
Chapter 5: Memory

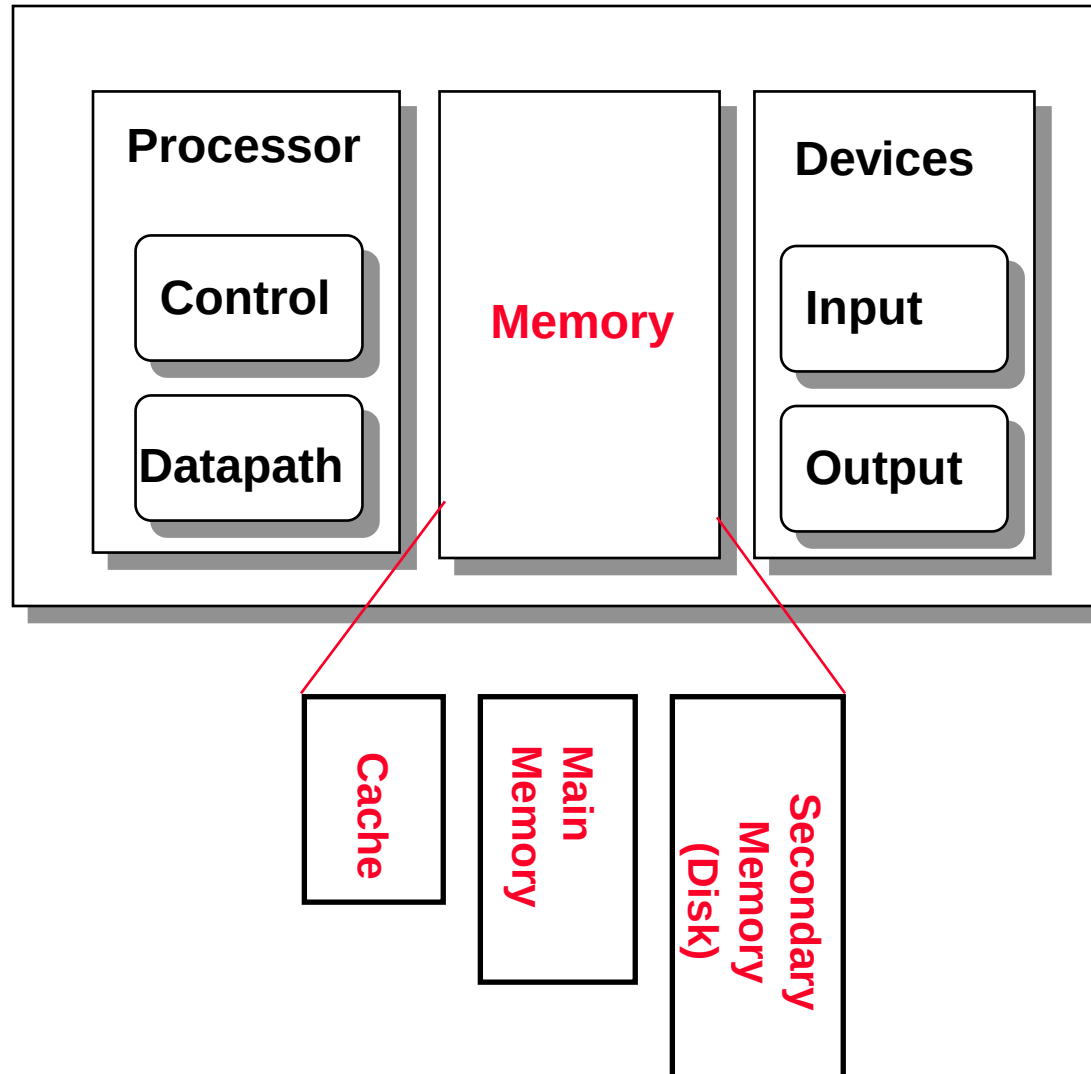
Ngo Lam Trung

[with materials from *Computer Organization and Design, 4th Edition*,
Patterson & Hennessy, © 2008, MK
and M.J. Irwin's presentation, PSU 2008]

Content

- ❑ Memory hierarchy
- ❑ Cache

Review: Major Components of a Computer



Memory technology

- ❑ Static RAM (SRAM)

- ❑ 0.5ns – 2.5ns, \$2000 – \$5000 per GB

- ❑ Dynamic RAM (DRAM)

- ❑ 50ns – 70ns, \$20 – \$75 per GB

- ❑ Magnetic disk

- ❑ 5ms – 20ms, \$0.20 – \$2 per GB

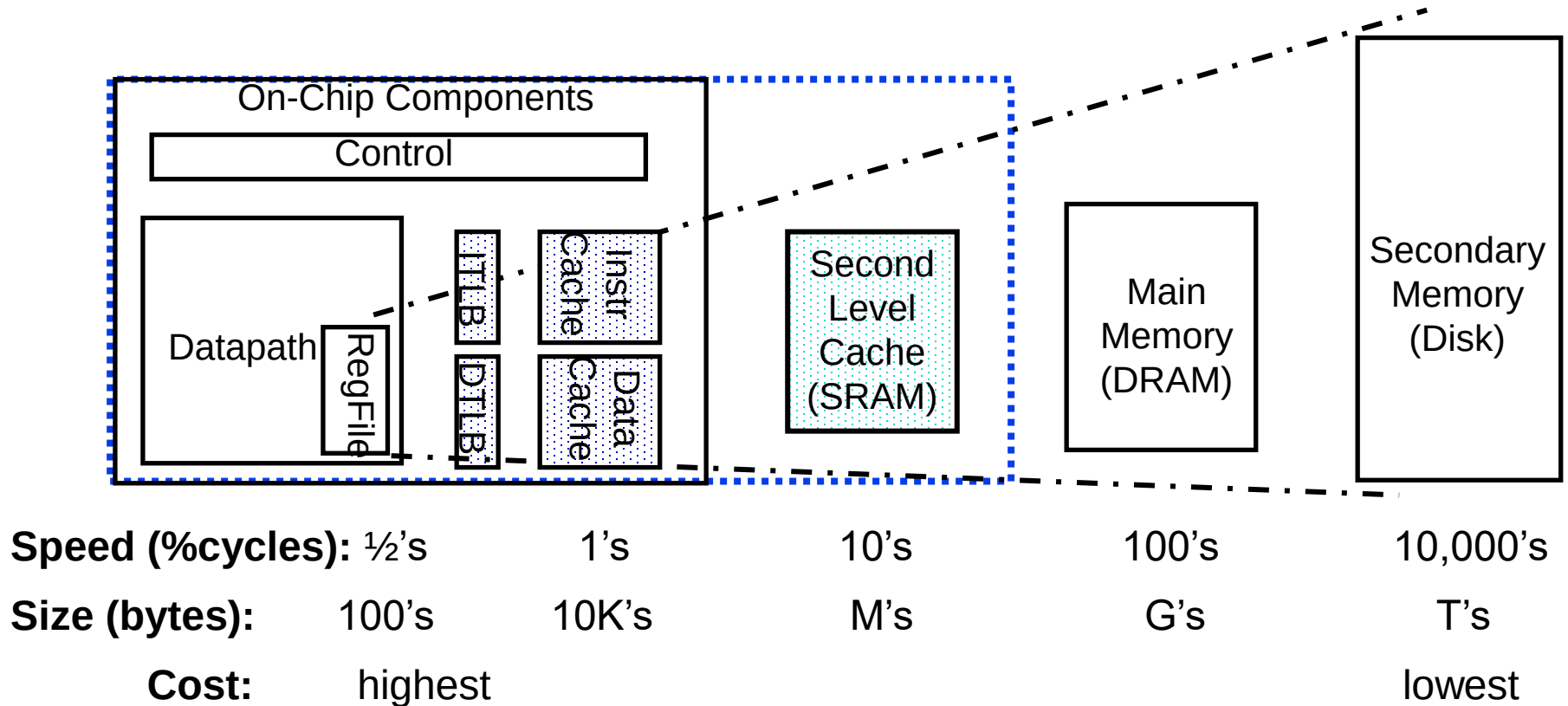
- ❑ Fact:

- ❑ Large memories are slow
 - ❑ Fast memories are small (and expensive)

- ❑ Ideal memory

- ❑ Access time of SRAM
 - ❑ Capacity and cost/GB of disk

A Typical Memory Hierarchy



- ❑ How to get an ideal memory
 - ❑ As fast as SRAM
 - ❑ As cheap as disk?

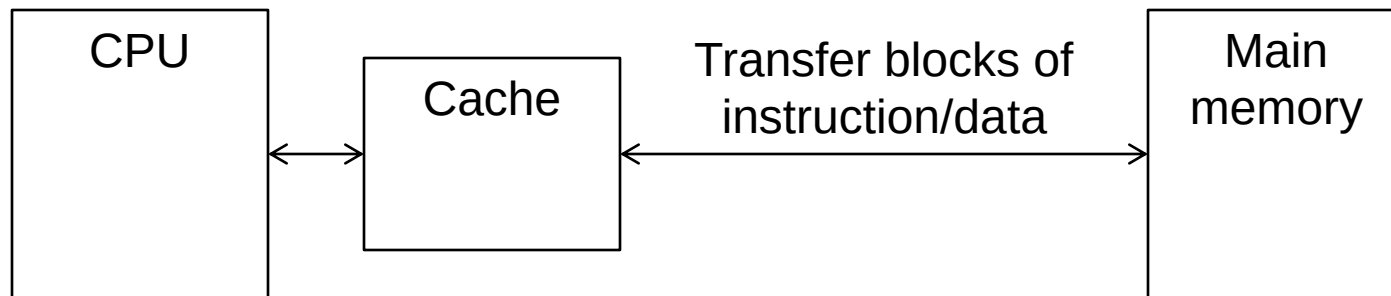
The Memory Hierarchy: Locality Principal

❑ C program

```
int x[1000], temp;
for (i = 0; i < 999; i++)
    for (j = i+1; j < 1000; j++)
        if (x[i] < x[j])
        {
            temp = x[i];
            x[i] = x[j];
            x[j] = temp;
        }
```

Data memory space of x is access multiple times

Instruction memory space of the two for loops are used repeatedly



Instruction fetch/data access
(multiple times)

The Memory Hierarchy: Locality Principal

❑ Temporal Locality (locality in time)

- ❑ If a memory location is referenced then it will tend to be referenced again soon

⇒ Keep **most recently accessed** data items closer to the processor

❑ Spatial Locality (locality in space)

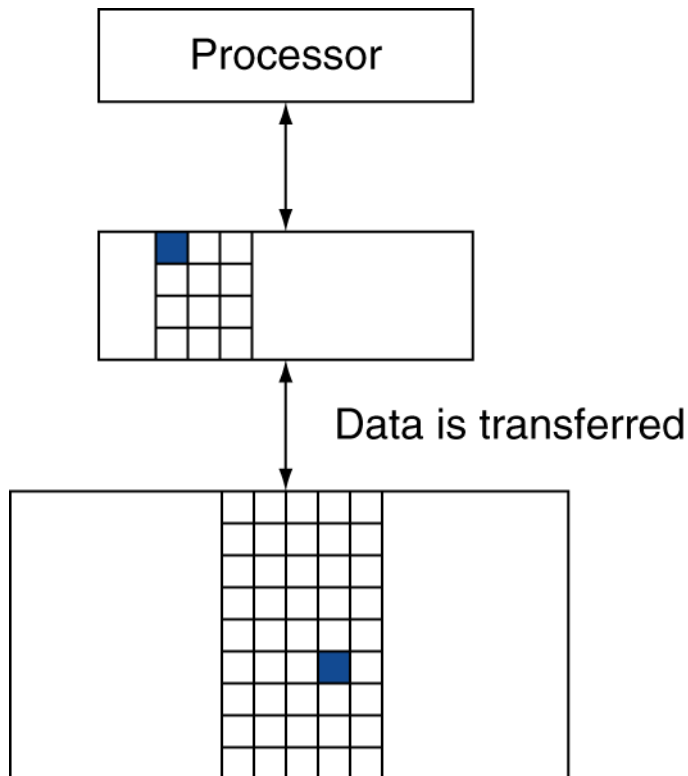
- ❑ If a memory location is referenced, the locations with nearby addresses will tend to be referenced soon

⇒ Move blocks consisting of **contiguous words** closer to the processor

Terminology

- ❑ **Block** (or line): the minimum unit of information that is present in a cache
- ❑ **Hit**: the requested memory item is found in a level of the memory hierarchy
- ❑ **Miss**: the requested memory item is *not* found in a level of the memory hierarchy

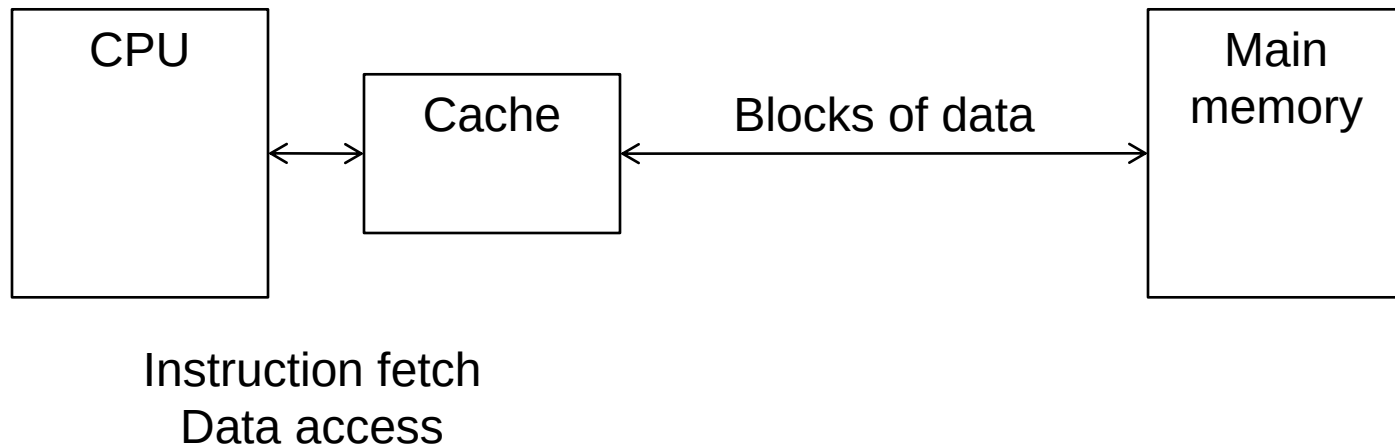
Hierarchical memory access



- ❑ Block: unit of copying
 - ❑ May be multiple words
- ❑ If accessed data is present in upper level
 - ❑ Hit: access satisfied by upper level
 - Hit ratio: hits/accesses
- ❑ If accessed data is absent
 - ❑ Miss: block copied from lower level
 - Time taken: miss penalty
 - Miss ratio: misses/accesses
 $= 1 - \text{hit ratio}$
 - ❑ Then accessed data supplied from upper level

Cache

- ❑ The memory hierarchy between the processor and main memory
 - ❑ CPU fetch instructions and data from cache, if found (**cache hit**) → fast access.
 - ❑ If not found (**cache miss**) → load from main memory into cache, then access in cache → miss penalty
- ❑ Cache is a small windows to see the main memory

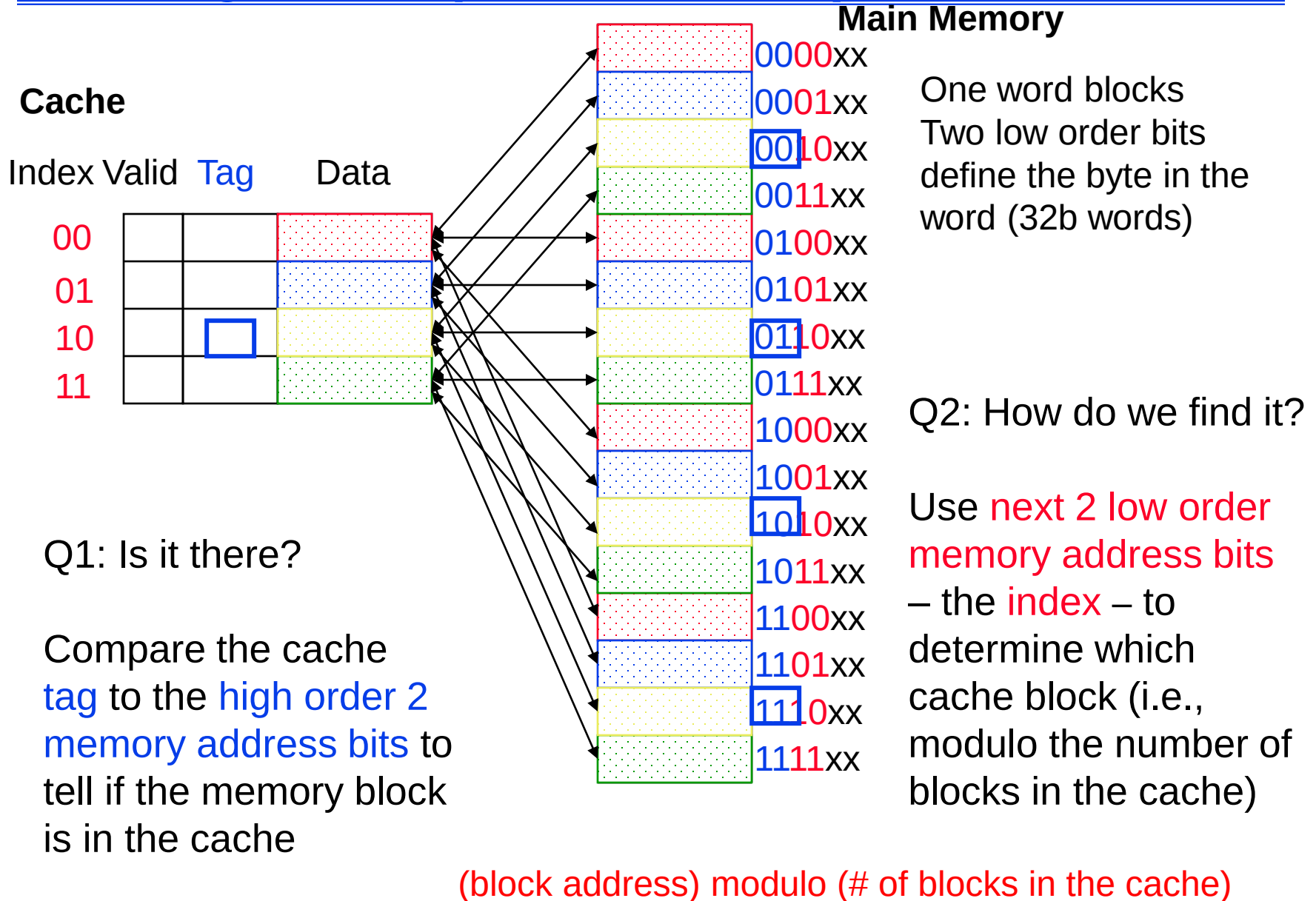


Cache Basics

- ❑ Two questions to answer (in hardware):
 - ❑ Q1: How do we know if a data item is in the cache?
 - ❑ Q2: If it is, how do we find it?

- ❑ Direct mapped
 - ❑ Each memory block is mapped to exactly one block in the cache
 - lots of lower level blocks must **share** blocks in the cache
 - ❑ Address mapping (to answer Q2):
 $(\text{block address}) \bmod (\# \text{ of blocks in the cache})$
 - ❑ The **tag** field: associated with each cache block that contains the address information (the upper portion of the address) required to identify the block (to answer Q1)
 - ❑ The valid bit: if there is data in the block or not

Caching: A Simple First Example



Direct Mapped Cache

❑ Consider the main memory word reference string

Start with an empty cache - all
blocks initially marked as not valid

0 1 2 3 4 3 4 15

0

1

2

3

4

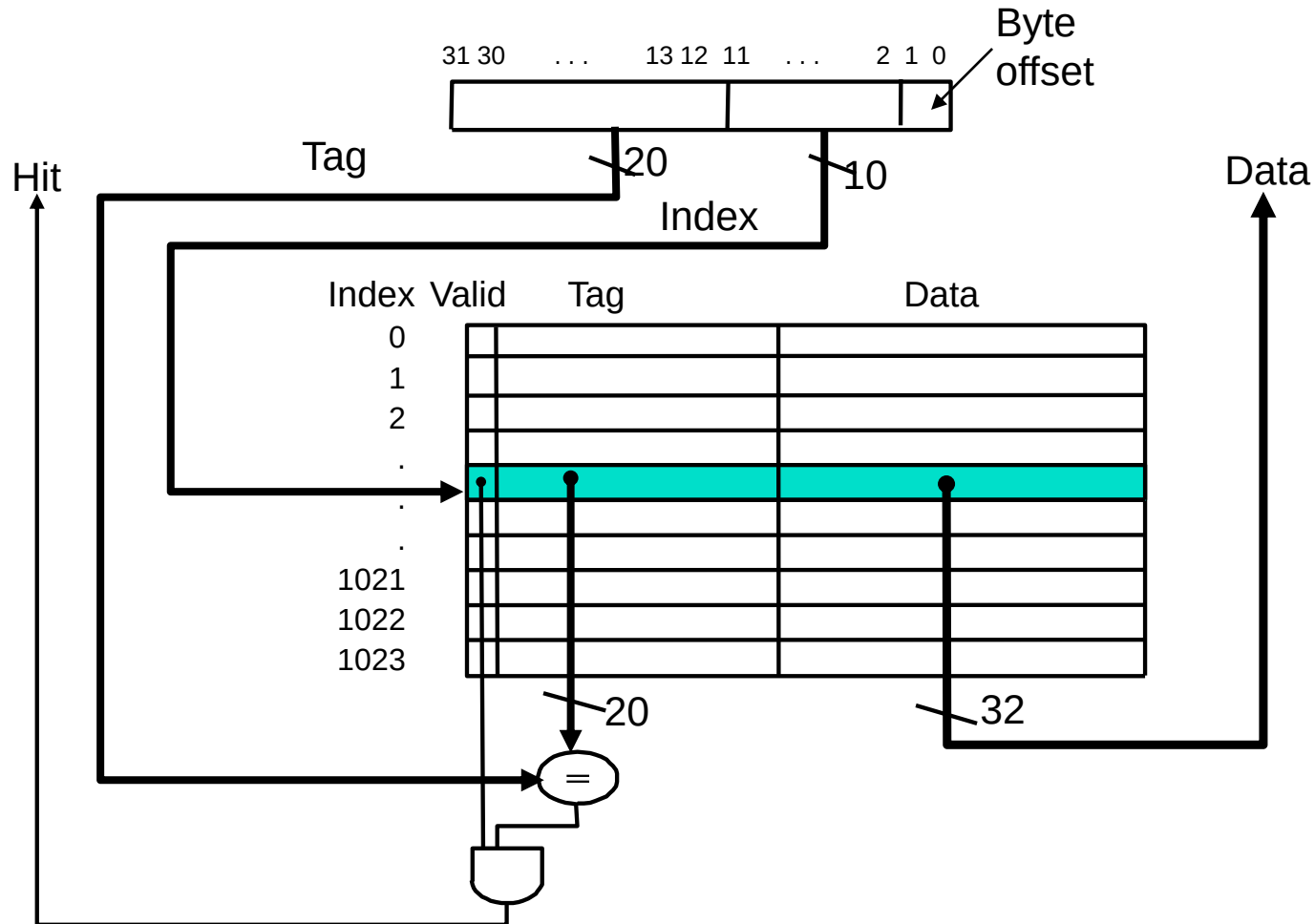
3

4

15

MIPS Direct Mapped Cache Example

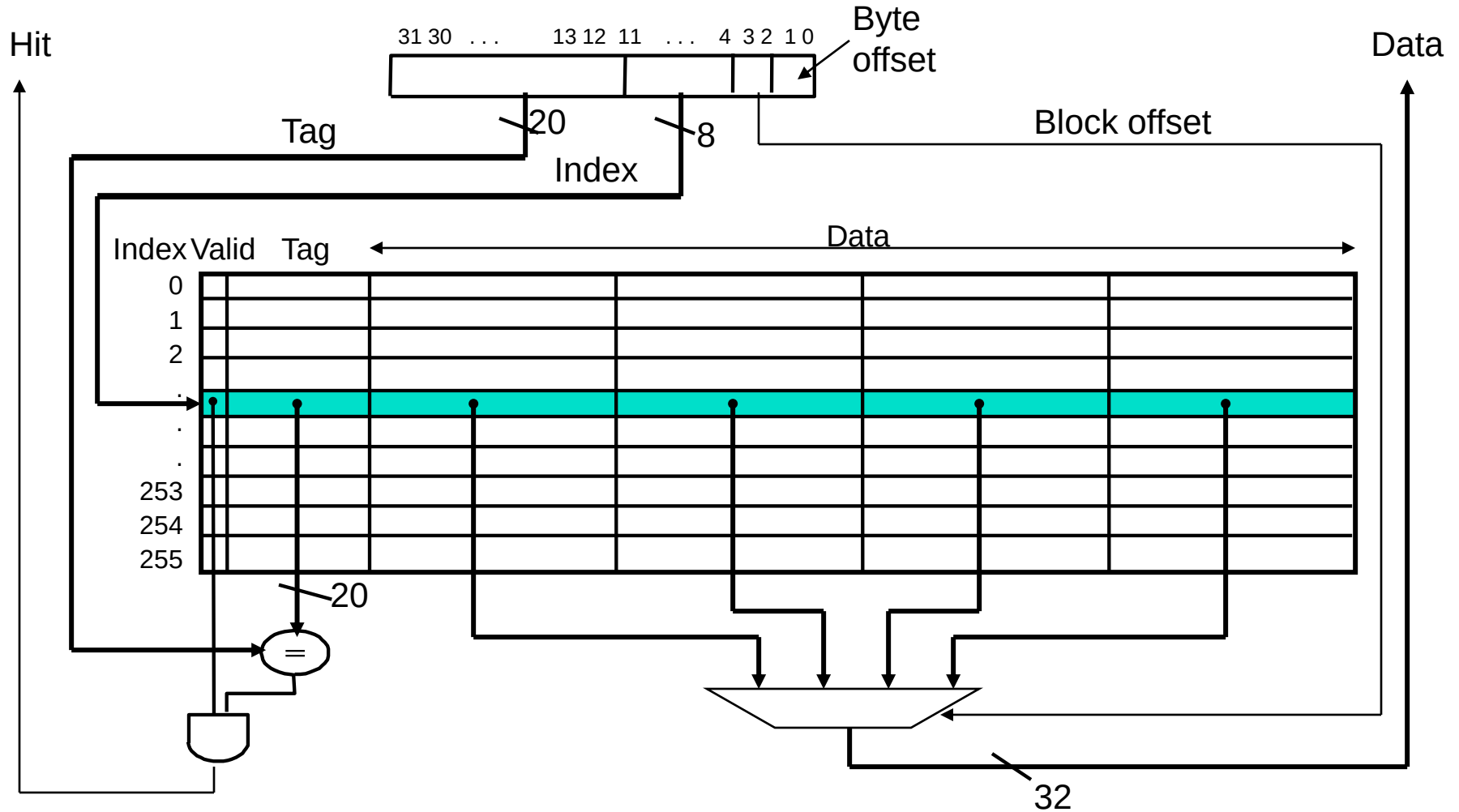
- ❑ One word blocks, cache size = 1K words (or 4KB)



What kind of locality are we taking advantage of?

Multiword Block Direct Mapped Cache

- ❑ Four words/block, cache size = 1K words



What kind of locality are we taking advantage of?

Taking Advantage of Spatial Locality

- ❑ Let cache block hold more than one word

Start with an empty cache - all
blocks initially marked as not valid

0 1 2 3 4 3 4 15

0

1

2

3

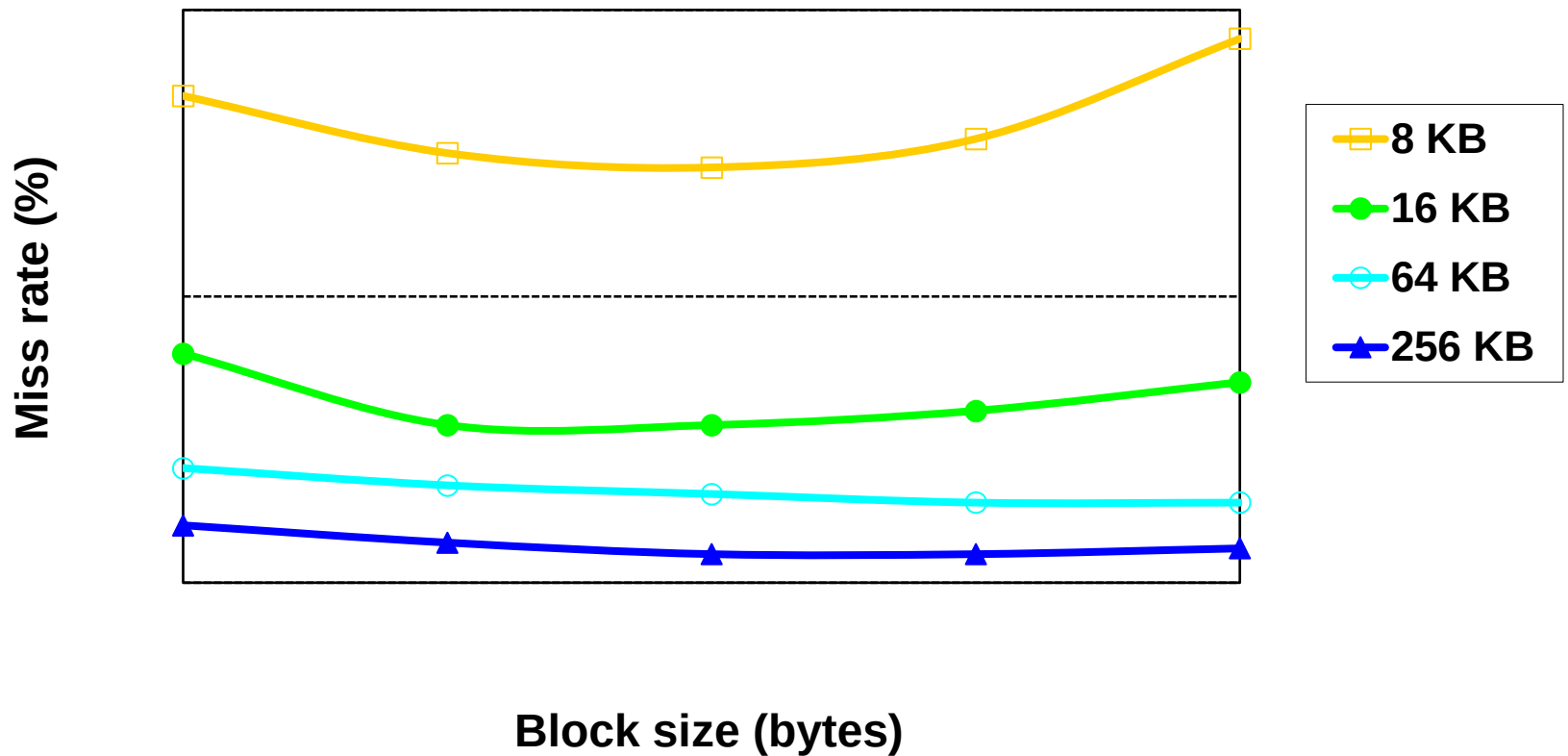
4

3

4

15

Miss Rate vs Block Size vs Cache Size



- ❑ Miss rate goes up if the block size becomes a significant fraction of the cache size because the number of blocks that can be held in the same size cache is smaller (increasing **capacity** misses)

Cache Field Sizes

- ❑ The number of bits in a cache includes both the storage for data and for the tags
 - ❑ 32-bit byte address
 - ❑ For a direct mapped cache with 2^n blocks, n bits are used for the index
 - ❑ For a block size of 2^m words (2^{m+2} bytes), m bits are used to address the word within the block and 2 bits are used to address the byte within the word
- ❑ What is the size of the tag field?
- ❑ The total number of bits in a direct-mapped cache is then
$$2^n \times (\text{block size} + \text{tag field size} + \text{valid field size})$$
- ❑ How many total bits are required for a direct mapped cache with 16KB of data and 4-word blocks assuming a 32-bit address?

Handling Cache Hits

❑ Read hits (I\$ and D\$)

- ❑ this is what we want!

❑ Write hits (D\$ only)

- ❑ require the cache and memory to be **consistent**
 - always write the data into both the cache block and the next level in the memory hierarchy (**write-through**)
 - writes run at the speed of the next level in the memory hierarchy – so slow! – or can use a **write buffer** and stall only if the write buffer is full
- ❑ allow cache and memory to be **inconsistent**
 - write the data only into the cache block (**write-back** the cache block to the next level in the memory hierarchy when that cache block is “evicted”)
 - need a **dirty** bit for each data cache block to tell if it needs to be written back to memory when it is evicted – can use a **write buffer** to help “buffer” write-backs of dirty blocks

Sources of Cache Misses

❑ Compulsory (cold start, first reference):

- ❑ First access to a block.
- ❑ We cannot do much on this.
- ❑ Solution: increase block size (but also increases miss penalty).

❑ Capacity:

- ❑ Cache cannot contain all blocks accessed by the program
- ❑ Solution: increase cache size (may increase access time)

❑ Conflict (collision):

- ❑ Multiple memory locations mapped to the same cache location
- ❑ Solution 1: increase cache size
- ❑ Solution 2: increase associativity (may increase access time)

Handling Cache Misses (Single Word Blocks)

❑ Read misses (I\$ and D\$)

- ❑ **stall** the pipeline, fetch the block from the next level in the memory hierarchy, install it in the cache and send the requested word to the processor, then let the pipeline resume

❑ Write misses (D\$ only)

1. **stall** the pipeline, fetch the block from next level in the memory hierarchy, install it in the cache (which may involve having to evict a dirty block if using a write-back cache), write the word from the processor to the cache, then let the pipeline resume

or

2. **Write allocate** – just write the word into the cache updating both the tag and data, no need to check for cache hit, no need to stall

or

3. **No-write allocate** – skip the cache write (but must invalidate that cache block since it will now hold stale data) and just write the word to the write buffer (and eventually to the next memory level), no need to stall if the write buffer isn't full

Multiword Block Considerations

❑ Read misses (I\$ and D\$)

- ❑ Processed the same as for single word blocks – a miss returns the entire block from memory
- ❑ Miss penalty grows as block size grows
 - **Early restart** – processor resumes execution as soon as the requested word of the block is returned
 - **Requested word first** – requested word is transferred from the memory to the cache (and processor) first
- ❑ **Nonblocking cache** – allows the processor to continue to access the cache while the cache is handling an earlier miss

❑ Write misses (D\$)

- ❑ If using write allocate must *first* fetch the block from memory and then write the word to the block (or could end up with a “garbled” block in the cache (e.g., for 4 word blocks, a new tag, one word of data from the new block, and three words of data from the old block))

Reducing Cache Miss Rates #1

1. Allow more flexible block placement
 - ❑ **Direct mapped cache:** a memory block maps to exactly one cache block
 - ❑ **Fully associative cache** allow a memory block to be mapped to *any* cache block
 - ❑ A compromise is to divide the cache into **sets** each of which consists of n “ways” (**n-way set associative**). A memory block maps to a unique set (specified by the index field) and can be placed in any way of that set (so there are n choices)
$$(\text{block address}) \bmod (\# \text{ sets in the cache})$$

Another Reference String Mapping

❑ Consider the main memory word reference string

Start with an empty cache - all
blocks initially marked as not valid

0 4 0 4 0 4 0 4

0

4

0

4

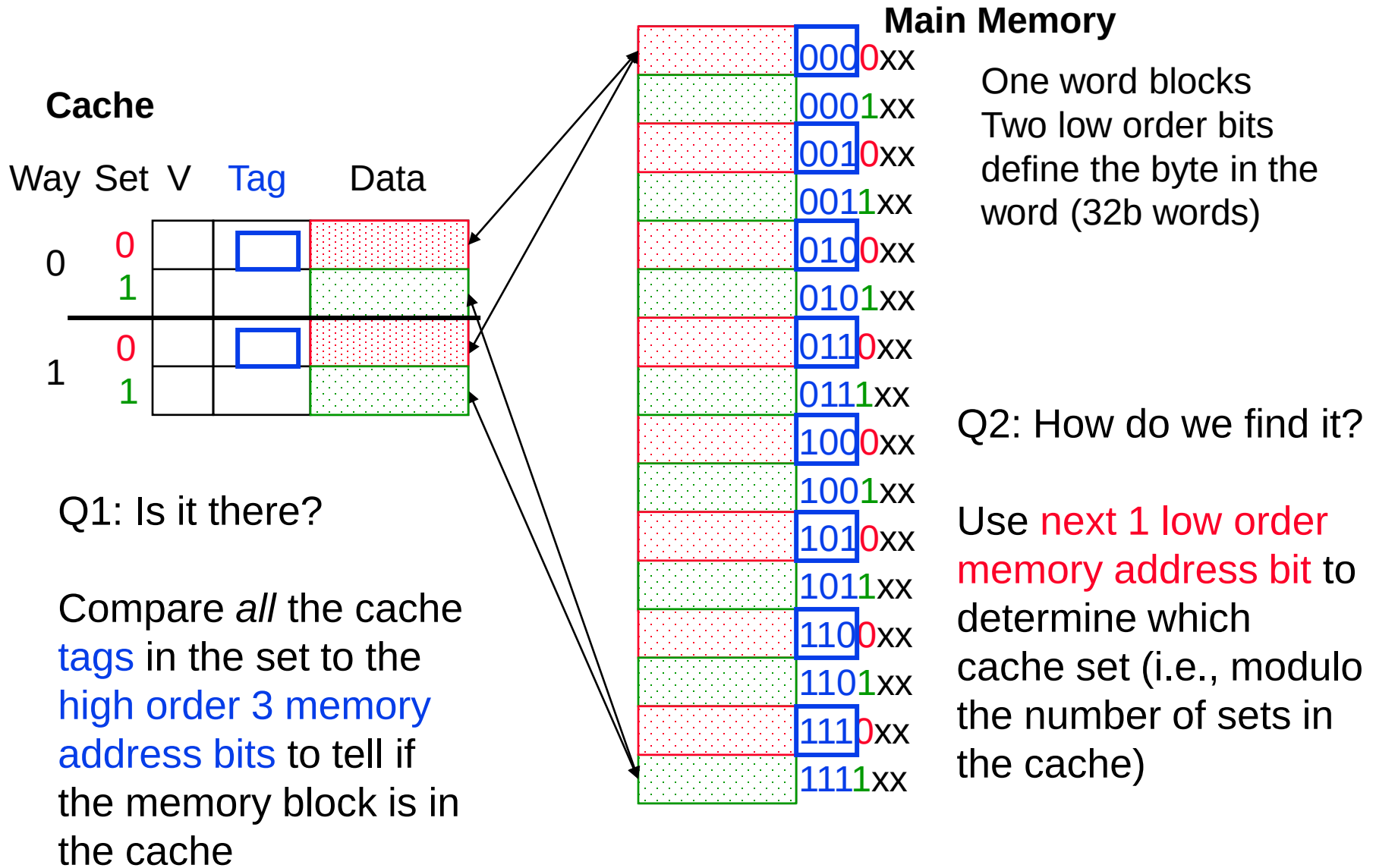
0

4

0

4

Set Associative Cache Example



Another Reference String Mapping

❑ Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 4 0 4 0 4 0 4

0 miss

4 miss

0 hit

4 hit

000	Mem(0)

000	Mem(0)
010	Mem(4)

000	Mem(0)
010	Mem(4)

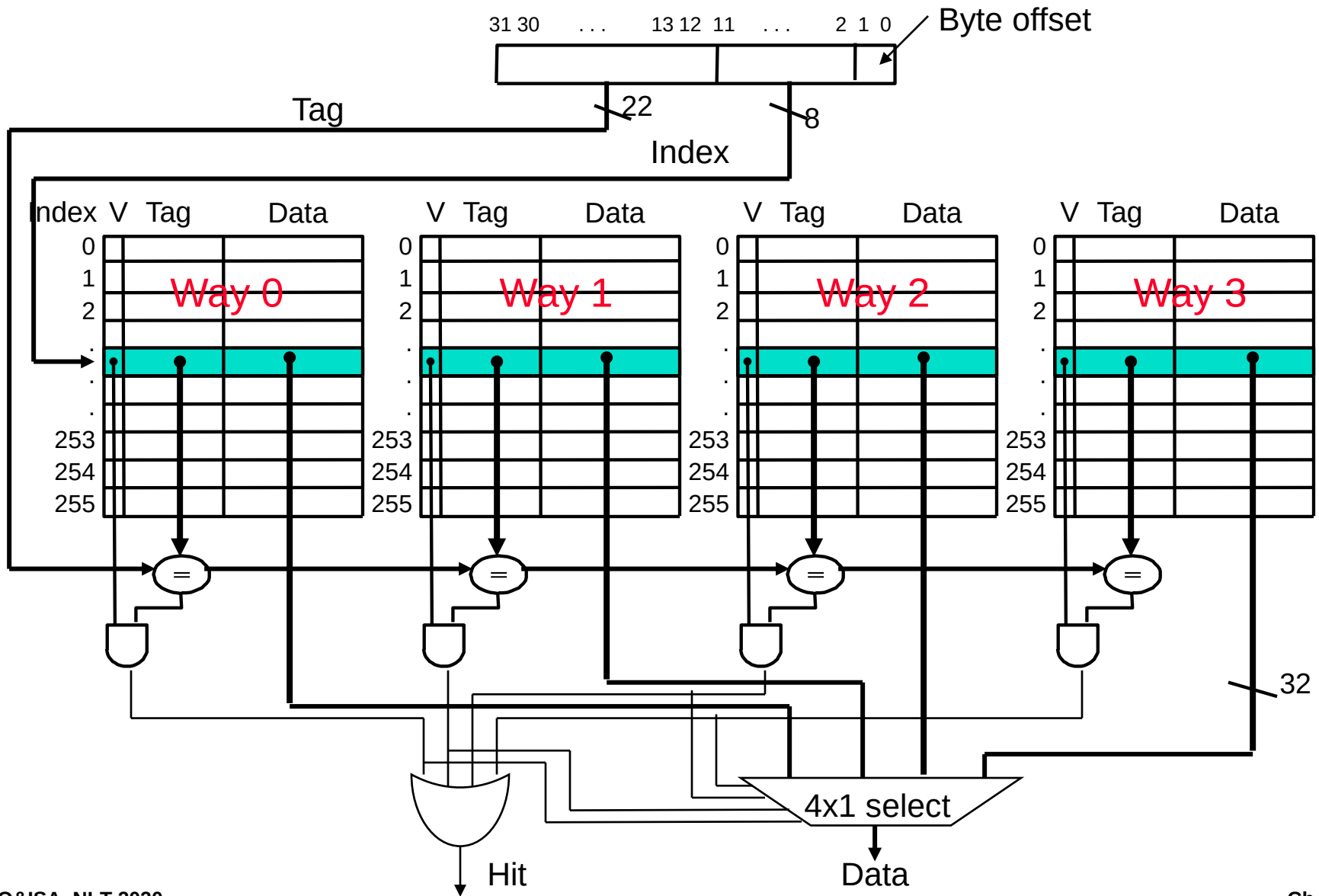
000	Mem(0)
010	Mem(4)

❑ 8 requests, 2 misses

❑ Solves the ping pong effect in a direct mapped cache due to **conflict** misses since now two memory locations that map into the same cache set can co-exist!

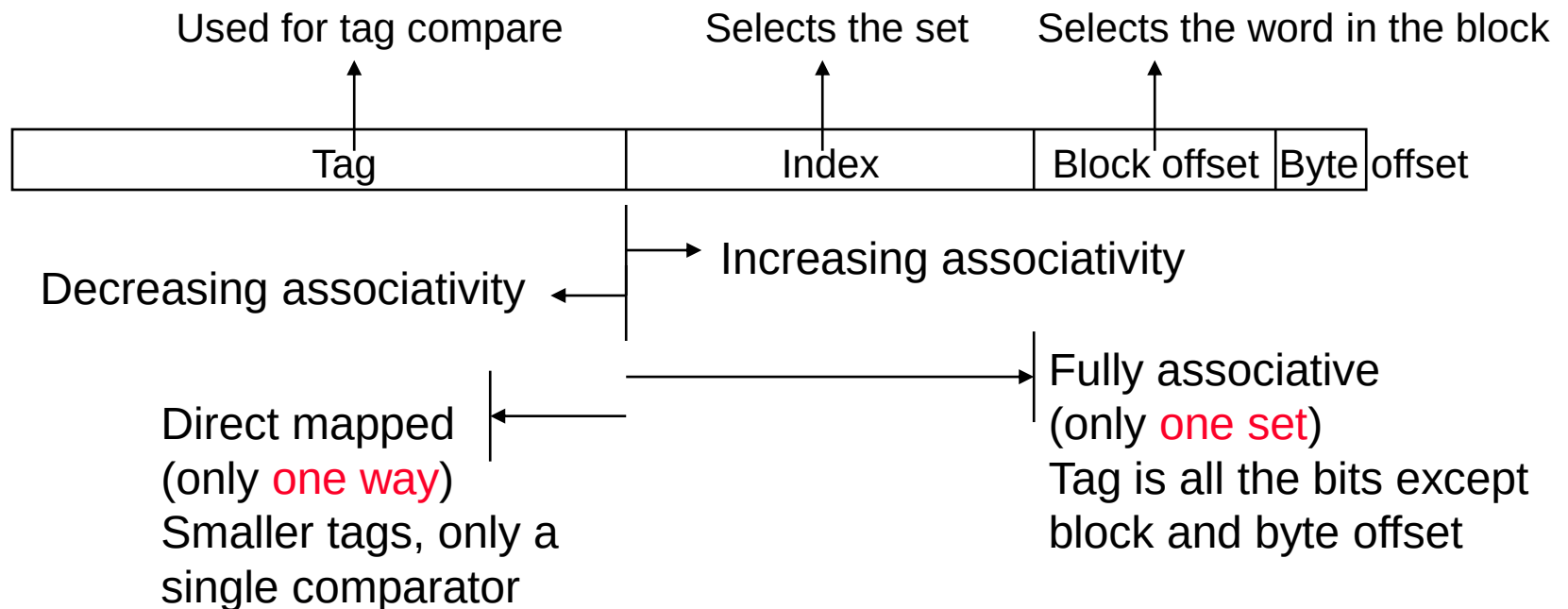
Four-Way Set Associative Cache

❑ $2^8 = 256$ sets each with four ways (each with one block)



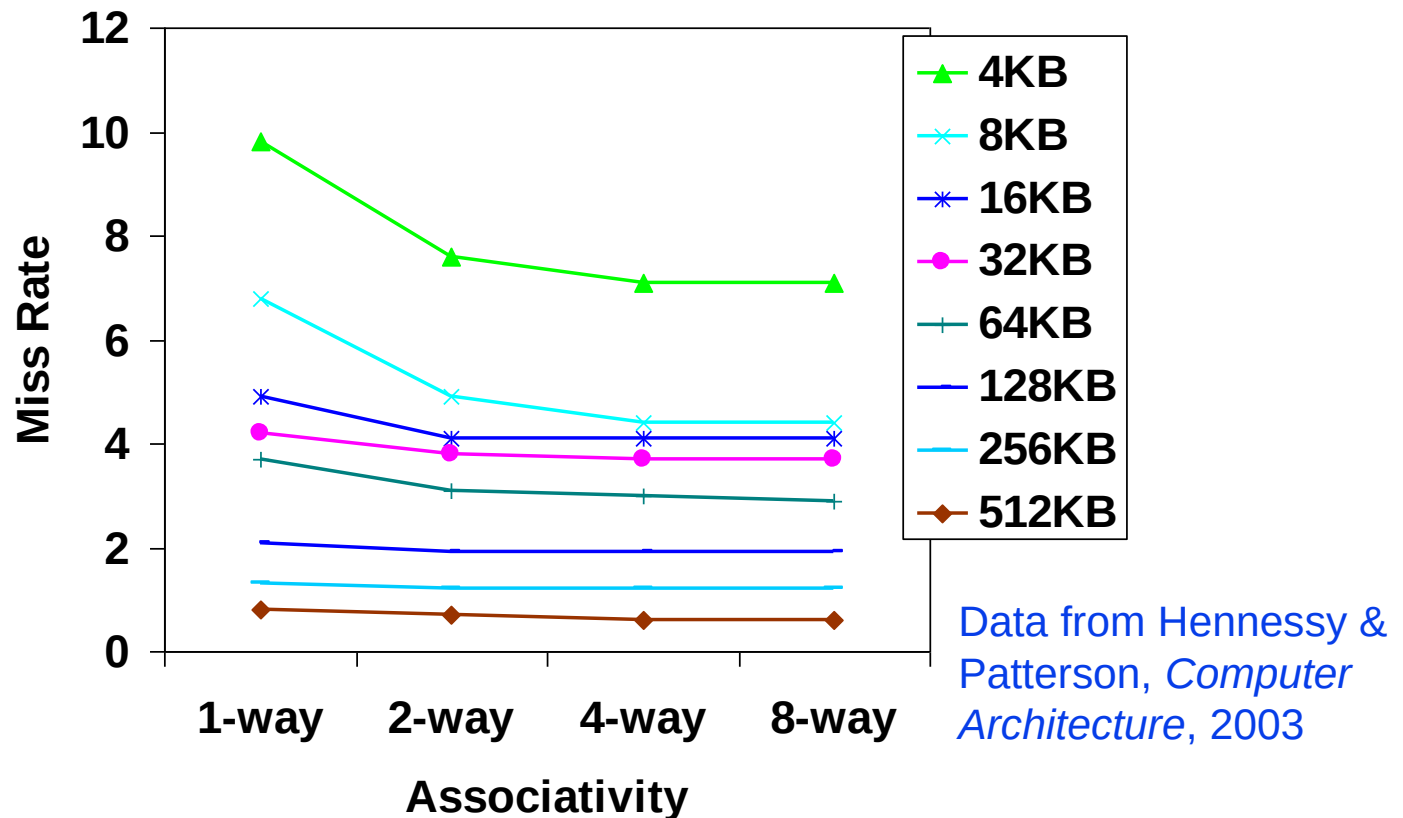
Range of Set Associative Caches

- ❑ For a fixed size cache, increase of the number of blocks per set results in decrease of the number of sets



Benefits of Set Associative Caches

- ❑ The choice of direct mapped or set associative depends on the cost of a miss versus the cost of implementation



- ❑ Largest gains are in going from direct mapped to 2-way (20%+ reduction in miss rate)

Reducing Cache Miss Rates #2

2. Use multiple levels of caches
 - ❑ Normally a **unified** L2 cache (holding both instructions and data) and in some cases even a unified L3 cache

Multilevel Cache Design Considerations

- ❑ Design considerations for L1 and L2 caches are very different
 - ❑ Primary cache should focus on **minimizing hit time** in support of a shorter clock cycle
 - Smaller with smaller block sizes
 - ❑ Secondary cache(s) should focus on **reducing miss rate** to reduce the penalty of long main memory access times
 - Larger with larger block sizes
 - Higher levels of associativity
- ❑ The miss penalty of the L1 cache is significantly reduced by the presence of an L2 cache – so it can be smaller but have a higher miss rate
- ❑ For the L2 cache, hit time is less important than miss rate
 - ❑ The L2\$ hit time determines L1\$'s miss penalty
 - ❑ L2\$ local miss rate >> than the global miss rate

Summary

- ❑ Memory hierarchy and the locality principal
- ❑ Cache design
 - ❑ Direct mapped
 - ❑ Set associative
 - ❑ Memory access when cache hit and miss